

Mortgage Engine Code Walkthrough

A maintainer's guide to the mortgage calculation flow, the field-presence solver, APR and APR-crossover comparison, and the underlying financial mathematics.

Scope. The mortgage calculation engine (`Mortgage.pas`), the screen layer and APR/row-generation dialogs that feed it (`MortgageScreenUnit.pas` and the `APR* / SelectAPRDlg / MortgageRowGenerationDlg` units), and the shared interest mathematics (`INTSUTIL.pas`).

Audience. Engineers maintaining or porting the financial logic. Every claim is anchored to a `file:line` reference for verification.

Authority. The DOS source is the authoritative implementation of the financial logic. Line numbers cite the annotated tree `legacy/src_documented/dos_source/`, which carries inline commentary on the routines documented here; `legacy/src/dos_source/` is the same code without comments (different line numbers).

Companion. This document follows the same structure as `amortization_dos_walkthrough.pdf`. The amortization, present-value, and actuarial flows share the same field-presence-dispatch pattern documented here.

Contents

0 How to read this document

1 Architecture & data model

1.1 The screen/engine split · 1.2 The mortgageline record · 1.3 The internal rate convention

2 The UI layer: cell → record + status

2.1 Grid & columns · 2.2 The blank-vs-typed mapping · 2.3 Mutual exclusions & validation · 2.4 The calc trigger

3 The core Calc dispatch

3.1 FirstPass · 3.2 Summation · 3.3 ComputeCashPctAndFinanced · 3.4 Price ↔ Monthly · 3.5 BalloonCalc

4 Balloon & early-termination math

5 APR: the held-to-term yield

6 APR comparison & the crossover solver

7 Row generation (what-if tables)

8 Maintainer notes & known quirks

9 Appendix: call graph & file:line index

0 How to read this document

The Per%Sense mortgage screen sizes a loan. Like every Per%Sense screen it uses field-presence dispatch: the user fills in what they know, leaves one field blank, and the engine solves for the blank. Leave the monthly payment blank to size a payment; leave the price blank to find the most expensive house a payment can buy; leave a balloon amount blank to size a balloon. On top of the single-loan calc sit two comparison features: a single-loan APR (the true yield once points are folded in) and a two-loan APR crossover (the holding period at which a low-rate/high-points loan stops beating a high-rate/low-points loan).

Citation convention

Inline references look like `Mortgage.pas:294`. Unless a range is given, the number is the line where the construct begins. All line numbers refer to the annotated tree `legacy/src_documented/dos_source/`; the un-annotated tree `legacy/src/dos_source/` holds the same code at different line numbers.

The cast of files

File	Role
<code>Mortgage.pas</code>	Engine. The single-row Calc dispatch, the Summation annuity factor, the APR and crossover solvers, and row generation.
<code>MortgageScreenUnit.pas</code>	UI. The grid, cell ↔ record marshalling, validation, the calc trigger, and the CompareAPRs / what-if entry points.
<code>SelectAPRDlgUnit / APRComparisonDLGUnit</code>	Pick the second loan to compare; display the comparison result.
<code>APROptionsDlgUnit / APRReportScreenUnit</code>	Collect cost adjustments and render the Reg-Z (Truth-in-Lending) APR report box.
<code>MortgageRowGenerationDlgUnit</code>	Collect the "vary this column" parameters for what-if table generation.
<code>INTSUTIL.pas</code>	Shared math: <code>exxp/lnn</code> , <code>YieldFromRate/RateFromYield</code> , <code>RealPerYr</code> . (Documented in detail in the amortization walkthrough, §5.)

The two things to internalize. (1) Summation (§3.2) is the annuity factor at the heart of every mortgage formula — the present value of n monthly payments of \$1. (2) The stored rate is a continuously-compounded rate, never the displayed percentage; the screen converts to/from the 12-period yield at the boundary. Miss either and every number drifts.

1 Architecture & data model

1.1 The screen/engine split

The mortgage feature is two layers over a shared model. The screen (`MortgageScreenUnit.pas`) owns a grid of cells and marshals them into a global array of `mortgageline` records; the engine (`Mortgage.pas`) reads those records, solves the blank field, and writes results back with an `outp` status. The screen never computes; the engine never decides what the user sees.

```

MortgageGrid cells --> AssignMortgageValuesEx --> e[1..maxlines]: ^mortgageline    (PEDATA.pas:254)
                                     |
                               DoCalculation --> CalculateRows(first,last)        (Mortgage.pas:1364)
                                     |
                               FirstPass --> Calc (field-presence dispatch)        (Mortgage.pas:235 / 294)
                                     |
                               TMortgage2Grid <-- results (status = outp)           (MortgageScreenUnit.pas:515)

Comparison overlay:  CompareAPRs --> ReportComparisonOfAPRs --> crossover solver (Mortgage.pas:945 / 556)
```

1.2 The mortgageline record

Every loan row is a mortgageline (alias mtgline) record `PETYPES.PAS:717–741`. Each numeric field is paired with an `inout` status field recording its provenance (empty=0 blank, `outp=1` computed, `defp=2` defaulted, `inp=3` user-typed — `PETYPES.PAS:178,215–218`):

Field	Screen label	Financial meaning
price	Price	Purchase price of the property.
points	Pts	Origination points as a fraction (0.02 = 2 points), charged on the financed amount.
pct	% Down	Down-payment fraction of price; 1–pct is the loan-to-value.
cash	Cash Required	Cash out of pocket = down payment + points = price·(pct + (1–pct)·points).
financed	Amount Borrowed	Loan principal = price·(1–pct).
years	Yrs	Term in years (integer).
rate	Rate	The true continuously-compounded rate (see §1.3). Displayed as a 12-period yield.
tax	Monthly Tax+Ins	Flat monthly escrow add-on — not amortized. The amortizing payment is monthly–tax.
monthly	Monthly Total	Total monthly payment = amortizing P&I + tax.
when / howmuch	Balloon Yrs / Amount	A balloon lump of howmuch due when years out.
balloonstatus	—	Derived enum (balloon_known, balloon_unk, balloon_blank) set by <code>FirstPass</code> (§3.1).

The grid columns are not one-to-one with fields: the three columns % Down / Cash Required / Amount Borrowed map to the mutually exclusive `pct` / `cash` / `financed` triad (§2.3). The global array is `e: mtgarray PEDATA.pas:254`, allocated per-row by the screen constructor. `ZeroMortgage Mortgage.pas:130` clears a row; `MortgageIsEmpty Mortgage.pas:163` tests whether all eleven status fields are empty.

1.3 The internal rate convention

Gotcha. `rate` is a continuous rate, not the displayed percentage. Internally every formula uses $e^{\pm \text{rate} \cdot t}$. The screen converts on the way in — `rate := RateFromYield(typed%/100, 12) MortgageScreenUnit.pas:418` — and on the way out — `YieldFromRate(rate, 12)×100 MortgageScreenUnit.pas:558`. Monthly compounding ($n=12$) is hard-coded throughout the mortgage unit. A port that stores the nominal yield in `rate` corrupts every formula.

The two conversions are exact inverses `INTSUTIL.pas:1612,1624`: `YieldFromRate(rr,n) = nn·(e^(rr/nn)–1)` and `RateFromYield(yy,n) = nn·ln(1+yy/nn)`, with `nn = RealPerYr(n) = 12` here. See the amortization walkthrough §5.6 for the full treatment.

2 The UI layer: cell → record + status

The form is `TMortgageScreen MortgageScreenUnit.pas:77`, hosting a single `MortgageGrid MortgageScreenUnit.pas:78` whose rows are loan records.

2.1 Grid & columns

Column indices are named constants `MortgageScreenUnit.pas:63–73` (column 0 is the row-number gutter):

```
PriceCol=1   PointsCol=2   PercentDownCol=3   CashRequiredCol=4   AmountBorrowedCol=5
YearsCol=6   LoanRateCol=7   MonthlyTaxCol=8   MonthlyTotalCol=9   BalloonYearsCol=10   BalloonAmountCol=11
```

The authoritative store is the global `e: mtgarray PEDATA.pas:254`; the grid is a view kept in sync by themarshallers below.

2.2 The blank-vs-typed mapping

`AssignMortgageValuesEx MortgageScreenUnit.pas:336` (wrapped by `AssignMortgageValues` with `Hardness=inp`) maps a cell to a field + status. Every column uses the same blank-vs-typed shape — here is `Price MortgageScreenUnit.pas:343–353`:

```
if( Value = '' ) then begin
  e[ARow+1].Price := 0;
  e[ARow+1].PriceStatus := empty;           { BLANK -> the solve request }
  MortgageGrid.SetCellHardness( ACol, ARow, empty );
end else begin
  e[ARow+1].Price := StringFormat2Double( Value, IsError );
  e[ARow+1].PriceStatus := Hardness;        { typed -> inp }
end;
```

Conversions on the way in: Points and % Down are stored as fractions (/100); the loan rate becomes a true rate (`RateFromYield(%/100, 12) MortgageScreenUnit.pas:418`); Years/When use `StringFormat2Int`; Tax is stored raw. The inverse `TMortgage2Grid MortgageScreenUnit.pas:515` formats each field back (Points/Pct ×100, rate via `YieldFromRate×100 MortgageScreenUnit.pas:558`) and colours the cell by status, so a computed (outp) value reads as such. Note the 1-based record vs 0-based grid: `e[ARow+1]` everywhere.

2.3 Mutual exclusions & validation

The down-payment triad is mutually exclusive. `MortgageGridCellBeforeEdit MortgageScreenUnit.pas:297–316` clears the other two of % Down / Cash / Amount-Borrowed (cells and records) when any one is entered, so the engine never sees a contradictory triad.

`MortgageGridVerifyCellString MortgageScreenUnit.pas:1400–1449` bounds the raw (pre-÷100) inputs: points in [0,10), % down in [−9,100), years in [0,100), rate in (−100,100), balloon years in [0,99). The high-rate advisory is a separate, softer check: a typed Loan Rate whose stored true rate exceeds 0.19835162342 (≈20% nominal) pops a warning but still computes `MortgageScreenUnit.pas:285`. The engine's own `FirstPass` adds semantic checks: years ≤ 0, a balloon amount given without a date, and financed > price (\$3.1).

2.4 The calc trigger

`Calculate` (or `Enter` in a cell) flushes the pending edit and calls `DoCalculation(Row) MortgageScreenUnit.pas:498–506`, which publishes the used-row count and invokes the engine for that one row:

```
nlines[MTGBlock] := MortgageGrid.GetUsedRowCount();
CalculateRows( Row+1, Row+1 ); { Mortgage.pas:1364 }
TMortgage2Grid( e[Row+1], MortgageGrid, Row ); { results back, status=outp }
```

CalculateRows(First,Last) [Mortgage.pas:1364](#) loops the row range, running FirstPass then Calc per row and bailing on errorflag. OnHardenValue [MortgageScreenUnit.pas:676](#) promotes a computed cell to inp (so the next calc treats it as given).

3 The core Calc dispatch

3.1 FirstPass

FirstPass(row) [Mortgage.pas:235–270](#) clears errorflag/overflowflag, then validates and derives balloonstatus for the row:

Condition	Effect	Line
years typed and ≤ 0	RecordError + message (term must be positive)	Mortgage.pas:245
when + howmuch both typed	balloonstatus := balloon_known	Mortgage.pas:253
when typed, howmuch blank	balloonstatus := balloon_unk (solve for it)	Mortgage.pas:255
howmuch typed, when blank	errorflag := true — must say WHEN	Mortgage.pas:257–261
neither	balloonstatus := balloon_blank	Mortgage.pas:262
financed > price	message + RecordError on the financed cell (no errorflag — still computes; a negative %Down is the inconsistency signal)	Mortgage.pas:263–267

3.2 Summation — the annuity factor

Summation(ei, r, t) [Mortgage.pas:197–216](#) is the present value, at loan date, of 12·t monthly \$1 payments discounted at continuous rate r:

```
if (abs(r) < teeny) then summation := 12 * t { zero-rate limit }
else begin
  last := exp(-r * t); { e^(-r*t) }
  f := exp(-r * twelfth); { e^(-r/12) one-month discount }
  summation := f * (1 - last) / (1 - f);
end;
```

$$S(r,t) = f \cdot (1 - e^{(-r \cdot t)}) / (1 - f) = \sum_{k=1..12t} f^k, \quad f = e^{(-r/12)}$$

A geometric series of $n = 12 \cdot t$ monthly discount factors, degenerating to the payment count $12 \cdot t$ as $r \rightarrow 0$ (the teeny branch). This factor converts a level amortizing payment (monthly-tax) to/from a present value.

3.3 ComputeCashPctAndFinanced

Given Price typed and exactly one of Pct/Cash/Financed typed, this nested proc `Mortgage.pas:307–356` derives the other two (status outp):

```
cash = price·(pct + (1-pct)·points)      financed = price·(1-pct)
from cash:    pct = (cash/price - points)/(1 - points)
from financed: pct = 1 - financed/price
```

A price smaller than teeny aborts with a "price too small" message `Mortgage.pas:312–316`; a resulting `pct ≥ 0.995` (essentially 100% down) is flagged via `RecordError` and the `pct` set to the error sentinel `Mortgage.pas:326 / 343`. Whichever field was input keeps `inp`; the two derived become `outp`.

3.4 Calc — Price ↔ Monthly

`Calc(row) Mortgage.pas:294` indexes the data array as `i := row + scrollpos[MTGblock]` and dispatches purely on the `*status = inp` flags `Mortgage.pas:377–419`:

1. If `pricestatus = inp`, run `ComputeCashPctAndFinanced` to fill the triad; abort on errorflag `Mortgage.pas:380–382`.
2. Only if a down-payment field *and* Years *and* Rate are all `inp` `Mortgage.pas:384` does any payment/price solve run. First fold in any known balloon: `balloonval := howmuch · e-(rate·when)` `Mortgage.pas:388`.
3. Then the inner dispatch:
 - **Price + Monthly both typed** → if `balloon_unk`, `BalloonCalc` solves the balloon; otherwise it is over-determined and the engine complains ("Leave PRICE or MONTHLY PAYMENT or ANY BALLOON AMOUNT blank") `Mortgage.pas:391–402`.
 - **Price typed, Monthly blank** → `monthly := (price·(1-pct) - balloonval) / Summation(rate,years) + tax` `Mortgage.pas:405–409`.
 - **Monthly typed, Price blank** (and not `balloon_unk`) → solve price from Pct (`price := ((monthly-tax)·S + balloonval)/(1-pct)`) or from Cash (`price := cash + (1-points)·((monthly-tax)·S + balloonval)`); error if neither is present; then re-run `ComputeCashPctAndFinanced` and set `pricestatus := outp` `Mortgage.pas:411–428`.

3.5 BalloonCalc

The nested `BalloonCalc Mortgage.pas:363–377` solves the unknown balloon. The residual = `financed (= price·(1-pct))` minus the PV of the level payments minus any other known balloon's PV (passed in `balloonval`) is the balloon's value as of the loan date; multiplying by `e(rate·when)` compounds it forward to the balloon date:

```
balloonval := price*(1-pct) - (monthly-tax)*Summation(e[i]^, rate, years) - balloonval;
howmuch    := balloonval * exxp(rate * when);
howmuchstatus := outp;
{ balloonstatus := balloon_known; This messes things up if you CALC twice. }
```

Quirk worth porting. `BalloonCalc` deliberately does *not* promote `balloonstatus` to `balloon_known` `Mortgage.pas:374` — leaving it `balloon_unk` means a second `CALC` re-solves identically rather than treating the just-computed balloon as fixed input.

4 Balloon & early-termination math

The APR machinery needs to value a loan paid off early. `TerminalBalloon(ei, t)` [Mortgage.pas:438](#) is the remaining balance t years out, including the payment due that date — the financed principal grown forward at rate, net of the payments made and any contractual balloon already paid by t .

`ValueOfPaymentsForTerminatedLoan(ei, r, t)` [Mortgage.pas:463](#) is the PV (at loan date, at trial rate r) of all cash if the loan terminates at t , discounting the terminal payoff at r so the APR solver can search over r .

5 APR: the held-to-term yield

`EnoughDataForAPR` [Mortgage.pas:775](#) gates the APR: Financed, Monthly, Rate and Years must each be present and real. `IterateToFindAPRofTerminatedLoan` [Mortgage.pas:495](#) is the secant solver (first guess rate + points/years, ≤ 20 steps) that finds the rate equating discounted cash flows to $\text{financed} \cdot (1 - \text{points})$; `IterateToFindAPR` [Mortgage.pas:537](#) wraps it for the held-to-term case ($t = \text{years} + 1/12$).

`OneMonthAPR` [Mortgage.pas:755](#) is the closed-form worst case — the APR if the whole loan is paid off when the first payment is due, with points amortized over that single month:

```
apr_rate := 12 * (1 + YieldFromRate(ei.rate, 12)/12) / (1 - ei.points);
OneMonthAPR := YieldFromRate(apr_rate, 12);    { corrected 2/94 }
```

6 APR comparison & the crossover solver

`ReportComparisonOfAPRs` [Mortgage.pas:945](#) recomputes each loan, writes the rate/points/years/APR table, then decides which loan wins. It runs the **dominance test** first [Mortgage.pas:995–998](#): if one loan's APR is $\leq$ the other at both the one-month and full-term horizons (with a strict win somewhere), that loan is "always better" — and the routine prints exactly that and stops, never running the crossover solver. This is the correct behavior even when the two loans have *different terms*: a mismatched-term comparison resolves to "always better" rather than a fabricated crossover.

DOS output reference (the M6 fix.) The Go port mirrors this routine's wording byte for byte. On a crossover, DOS emits [Mortgage.pas:1001–1006](#):

```
timestr := Concat(strb(years,0), ' years');
if (months <> 0) then timestr := Concat(timestr, ', ', strb(months,0), ' months');
... ' APR's cross at ', 100*apr:7:4, ' for duration ', timestr, '.'
```

Both units use the plural literal — "1 years, 1 months" is the genuine DOS output. The "always better" branches and this crossover branch are the two end-states a port must reproduce exactly.

`IterateToFindCrossoverAPRAndTime` [Mortgage.pas:556](#) solves simultaneously for the rate and time at which both loans price to par (residual $1 - \text{ValueOfPayments}/(\text{financed} \cdot (1 - \text{points}))$), via a 2×2 finite-difference Newton step inverted by Cramer's rule, ≤ 40 iterations (`maxcount = 40` [Mortgage.pas:558](#)). When a balloon makes the value function discontinuous and Newton stalls, the nested `TryBalloonDates` [Mortgage.pas:646](#) samples each balloon loan's APR just before and after its balloon date and pins the crossover there if the ordering flips.

7 Row generation (what-if tables)

EnoughDataForRowGeneration [Mortgage.pas:1075–1078](#) returns true iff the row produced at least one outp field (Price, Monthly, or balloon HowMuch) — the row is fully solved and can be copied/incremented:

```
EnoughDataForRowGeneration :=
  ((e.pricesstatus = outp) or (e.monthlystatus = outp) or (e.howmuchstatus = outp));
```

The table is generated by CopyAndIncrement(basei, j) [Mortgage.pas:1218](#), which recurses over up to three varied columns. Its key detail: the rate column is stepped in *yield* space, not continuous-rate space — it converts the base rate to a yield, adds increment · count in percent, then converts back with RateFromYield [Mortgage.pas:1228–1233](#):

```
if (qcol[j] = mratecol) then basex := YieldFromRate(basex, 12);
...
x := basex + 0.01 * count * increment[j];
if (qcol[j] = mratecol) then x := RateFromYield(x, 12);
```

The Go bumpField reproduces this exactly so a "+0.5%" step means 0.5% of nominal rate.

8 Maintainer notes & known quirks

- **1-based records, 0-based grid.** Every screen access is e[ARow+1]; the engine adds scrollpos[MTGblock] on top. Off-by-one here silently mixes rows.
- **The triad is enforced before the engine sees it.** MortgageGridCellBeforeEdit guarantees at most one of %Down/Cash/Financed is non-blank, so Calc never has to break a tie.
- **financed > price is a warning, not an error.** FirstPass records the cell but does not set errorflag; the calc proceeds and yields a negative %Down/Cash, which is the meaningful "inconsistent inputs" signal.
- **The high-rate threshold is a UI advisory.** 0.19835162342 ≈ 20% nominal; it only warns, never blocks, and fires only on typed input [MortgageScreenUnit.pas:285](#).
- **BalloonCalc leaves the balloon "unknown" on purpose** so repeated CALCs are idempotent [Mortgage.pas:374](#).

9 Appendix: call graph & file:line index

Calculate / Enter -> DoCalculation(Row)	MortgageScreenUnit.pas:498
\ - CalculateRows(Row+1, Row+1)	Mortgage.pas:1364
- FirstPass (validate + balloon classification)	Mortgage.pas:235
\ - Calc (field-presence dispatch)	Mortgage.pas:294
- ComputeCashPctAndFinanced (triad)	Mortgage.pas:307
- Summation $f \cdot (1 - \text{last}) / (1 - f)$	Mortgage.pas:197
\ - solve Monthly Price BalloonCalc	Mortgage.pas:405 / 411 / 363
\ - TMortgage2Grid (record -> grid, status colouring)	MortgageScreenUnit.pas:515
CompareAPRs -> ReportComparisonOfAPRs	MortgageScreenUnit / Mortgage.pas:945
- IterateToFindAPR / OneMonthAPR (dominance)	Mortgage.pas:537 / 755
\ - IterateToFindCrossoverAPRandTime (2-var Newton)	Mortgage.pas:556
\ - TryBalloonDates (discontinuity fallback)	Mortgage.pas:646
What-if table -> CopyAndIncrement (recursive)	Mortgage.pas:1218
guard: EnoughDataForRowGeneration	Mortgage.pas:1075

Symbol	Reference
mortgageline record / status enum	PETYPES.PAS:717 · PETYPES.PAS:215
e: mtgarray	PEDATA.pas:254
FirstPass / Calc / Summation	Mortgage.pas:235 / 294 / 197
ComputeCashPctAndFinanced / BalloonCalc	Mortgage.pas:307 / 363
ReportComparisonOfAPRs / crossover / TryBalloonDates	Mortgage.pas:945 / 556 / 646
CopyAndIncrement / EnoughDataForRowGeneration	Mortgage.pas:1218 / 1075
RateFromYield / YieldFromRate	INTSUTIL.pas:1624 / 1612